

NumPy & Pandas Coding Assignment

Instructions

- Use **Python (NumPy + Pandas)**.
- Write clean and readable code with comments.
- Perform all tasks in a **Jupyter/Colab notebook**.
- Use **vectorized operations** wherever possible (avoid loops unless necessary).
- Display intermediate outputs for verification.

Q1. Employee Salary Adjustment

A company stores salaries in a NumPy array.

```
salaries = np.array([45000, 52000, 61000, 48000, 75000])
```

Tasks:

1. Increase all salaries by **10%**.
2. Find the **average salary**.
3. Find the **maximum and minimum salary**.
4. Print salaries **greater than 60000**.

Q2. Sales Data Analysis

Daily sales of a store for 10 days:

```
sales = np.array([120,150,180,200,170,160,210,220,190,175])
```

Tasks:

1. Compute **total sales**.
2. Compute **mean and standard deviation**.
3. Identify **days where sales exceeded the average**.
4. Normalize the sales using **Min-Max scaling**.

Q3. Matrix Operations for Business Data

A company has sales data for **3 products** across **4 regions**.

```
sales_matrix = np.array([  
    [120,140,130,150],  
    [80,90,100,95],  
    [200,210,190,205]  
])
```

Tasks:

1. Find **total sales per product**.
2. Find **total sales per region**.
3. Identify the **best performing product**.
4. Identify the **region with highest sales**.

Q4. Employee Dataset Exploration

Create a DataFrame:

```
data = {  
    'Employee':['A','B','C','D','E','F'],  
    'Department':['HR','IT','IT','Finance','HR','Finance'],  
    'Salary':[50000,65000,70000,72000,52000,68000],  
    'Experience':[2,5,7,6,3,4]  
}
```

Tasks:

1. Convert this dictionary into a **DataFrame**.
2. Display **basic statistics** of the dataset.
3. Find the **average salary per department**.

4. Identify employees with **experience greater than 4 years**.
 5. Add a new column **Bonus = 10% of salary**.
-

Q5. Missing Data Handling

Dataset:

```
data = {  
'Name':['John','Anna','Peter','Linda','David'],  
'Age':[28, np.nan, 35, 32, np.nan],  
'Salary':[50000,54000,np.nan,58000,60000]  
}
```

Tasks:

1. Detect **missing values**.
 2. Replace missing **Age** with **mean age**.
 3. Replace missing **Salary** with **median salary**.
 4. Remove rows with **any remaining missing values**.
-

Q6. Sales Performance Analysis

Dataset:

```
sales_data = {  
'Region':['North','South','East','West','North','South','East','West'],  
'Salesperson':['A','B','C','D','E','F','G','H'],  
'Sales':[12000,15000,13000,17000,16000,14000,15500,16500]  
}
```

Tasks:

1. Find **total sales per region**.
2. Identify the **top salesperson**.

3. Compute **average sales across regions**.
 4. Sort the dataset by **sales descending**.
-

Q7. Customer Purchase Analysis

Dataset:

```
data = {  
'CustomerID':[101,102,103,104,105,106,107],  
'Age':[25,45,35,50,23,40,60],  
'PurchaseAmount':[250,600,400,800,150,700,900],  
'City':['Delhi','Mumbai','Delhi','Chennai','Mumbai','Delhi','Chennai']  
}
```

Tasks:

1. Find the **average purchase amount by city**.
2. Identify customers with **purchase above overall average**.
3. Find the **city generating the highest revenue**.
4. Create a column **Age_Group**:
 - <30 → Young
 - 30–50 → Adult
 - 50 → Senior
5. Find **average purchase per age group**.

Q8. Time Series Sales Analysis

Generate data:

```
dates = pd.date_range('2024-01-01', periods=12, freq='M')
```

```
sales = [200,220,250,270,300,320,350,360,380,400,420,450]
```

```
df = pd.DataFrame({'Date':dates,'Sales':sales})
```

Tasks:

1. Convert Date to **datetime** index.
2. Compute **monthly percentage growth**.
3. Calculate **3-month rolling average sales**.
4. Identify the **month with highest growth rate**.

